COMPSCI 689
Lecture 15: Probabilistic Classifiction (and Beyond)

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Discrete Random Variables

A random variable Z taking values $z \in \mathcal{Z}$ is discrete if its probability distribution is defined by a probability mass function $P : \mathcal{Z} \to \mathbb{R}$ that satisfies the requirements below:

- Normalization: $\sum_{z \in \mathcal{Z}} P(Z = z) = 1$
- Non-Negativity: $\forall z \in \mathcal{Z}\ P(Z = z) \geq 0$

Parametric Probability Mass Functions

- A parametric probability mass function uses a set of parameters to provide access to a family of related probability mass functions.
- We will denote a parametric probability mass function by $P_\phi(Z = z)$ with parameters ϕ.
- We will denote by Φ the parameter space.
- To be valid, a parametric mass function must satisfy normalization and non-negativity for all $\phi \in \Phi$

Example: General Bernoulli Random Variables

Consider a biased coin. Let ϕ be the probability that the coin comes up heads. Let $z \in \{0, 1\}$ be the value of the coin flip (1 for heads, 0 for tails). We thus have:

- Values: $\mathcal{Z} = \{0, 1\}$
- Parameters: $\phi \in [0, 1]$
- Parameter Space: $\Phi = [0, 1]$
- Probability Mass Function: $P_\phi(Z = z) = \phi^z(1 - \phi)^{(1-z)}$

Example: General Categorical Random Variables

Suppose we have a die with C sides. Each side potentially comes up with a different probability. This is a general categorical random variable:

- Values: $\mathcal{Z} = \{1, 2, ..., C\}$
- Parameters: For each c we have $\phi_c \geq 0$ and $\sum_{c=1}^{C} \phi_c = 1$
- Parameter Space: $\Phi = \mathcal{S}$
- Mass Function: $P(Z = z|\phi) = \prod_{c=1}^{C} \phi_c^{[z=c]}$

Probabilistic Classification

- In probabilistic classification, our goal is to model the true probability distribution of the outputs $y \in \mathcal{Y}$ given the inputs $\mathbf{x} \in \mathcal{X}$.
- Specifically, we model $P_*(Y = y|\mathbf{X} = \mathbf{x})$ with a conditional parametric probability mass function $P_\theta(Y = y|\mathbf{X} = \mathbf{x})$.

Learning Probabilistic Classification Models

- So long as all model components are differentiable functions, we can learn the model parameters θ by minimizing the conditional negative log likelihood function given a data set $\mathcal{D}$:

$$nll(\mathcal{D}, \theta) = -\sum_{n=1}^{N} \log P_\theta(Y = y|\mathbf{X} = \mathbf{x})$$

Example: Probabilistic Binary Logistic Regression

- Suppose that $\mathcal{Y} = \{-1, 1\}$ and $\mathcal{X} \in \mathbb{R}^D$.
- Base Model: $P_\phi(Y = y) = \phi^{[y=1]}(1-\phi)^{[y=-1]}$
- Conditional Model:

$$P_\theta(Y = y|\mathbf{X} = \mathbf{x}) = f_\theta(\mathbf{x})^{[y=1]}(1 - f_\theta(\mathbf{x}))^{[y=-1]}$$

- Parameter Prediction Function: $f_\theta(\mathbf{x}) = \sigma(\mathbf{x}\theta) = \frac{1}{1+\exp(-\mathbf{x}\theta)}$
- Model Parameters: θ
- Negative Log Likelihood:

$$nll(\mathcal{D}, \theta) = -\sum_{n=1}^{N} \log P_\theta(Y = y_n|\mathbf{X} = \mathbf{x}_n)$$

Example: Probabilistic Binary Logistic Regression

$$
\begin{aligned}
nll(\mathcal{D}, \theta) &= -\sum_{n=1}^{N} \log P_\theta(Y = y_n | \mathbf{X} = \mathbf{x}_n) \\
&= -\sum_{n=1}^{N} \log \left(f_\theta(\mathbf{x}_n)^{[y_n=1]} (1 - f_\theta(\mathbf{x}_n))^{[y_n=-1]} \right) \\
&= -\sum_{n=1}^{N} \left([y_n = 1] \log f_\theta(\mathbf{x}_n) + [y_n = -1] \log(1 - f_\theta(\mathbf{x}_n)) \right)
\end{aligned}
$$

Example: Probabilistic Binary Logistic Regression

- Suppose that instead of defining $\mathcal{Y} = \{-1, 1\}$, we choose $\mathcal{Y} = \{0, 1\}$.
- In this case, the negative log likelihood can be written as:

$$\begin{aligned} nll(\mathcal{D}, \theta) &= -\sum_{n=1}^{N} \log \left(f_\theta(\mathbf{x}_n)^{y_n} (1 - f_\theta(\mathbf{x}_n))^{1-y_n} \right) \\ &= -\sum_{n=1}^{N} \left(y_n \log f_\theta(\mathbf{x}_n) + (1 - y_n) \log(1 - f_\theta(\mathbf{x}_n)) \right) \end{aligned}$$

- This function is referred to as the *binary cross entropy loss*.
- Minimizing the logistic loss under ERM and either version of the probabilistic binary logistic regression NLL function lead to equivalent optimization problems.

Multiclass Logistic Regression

- Suppose that $\mathcal{Y} = \{1, ..., C\}$ and $\mathcal{X} \in \mathbb{R}^D$.
- Base Model: $P_\phi(Y = y) = \prod_{c=1}^{C} \phi_c^{[y=c]}$
- Conditional Model: $P_\theta(Y = y|\mathbf{X} = \mathbf{x}) = \prod_{c=1}^{C} f_{\theta,c}(\mathbf{x})^{[y=c]}$
- Parameter Prediction Function:

$$f_{\theta,c}(\mathbf{x}) = \text{softmax}(\mathbf{x}, c, \theta) = \frac{\exp(\mathbf{x}\mathbf{w}_c)}{\sum_{k=1}^{C} \exp(\mathbf{x}\mathbf{w}_k)}$$

- Model Parameters: $\theta = [\mathbf{w}_1, ..., \mathbf{w}_C]$

Multiclass Logistic Regression

- Putting all of this together, we have the model:

$$P_\theta(Y = y|\mathbf{x}) = \prod_{c=1}^{C} \left(\frac{\exp(\mathbf{x}\mathbf{w}_c)}{\sum_{k=1}^{C} \exp(\mathbf{x}\mathbf{w}_k)} \right)^{[y=c]}$$

- There is one weight vector $\mathbf{w}_c$ per class (assuming bias absorption).
- Note that this parameterization is actually redundant due to the normalization constraint. This redundancy can optionally be removed by asserting that $\mathbf{w}_c = 0$ for one of the C classes, but generally does not cause issues.

Multiclass Logistic Regression

- Given the specification of the model:

$$P_\theta(Y = y|\mathbf{x}) = \prod_{c=1}^{C} \left(\frac{\exp(\mathbf{x}\mathbf{w}_c)}{\sum_{k=1}^{C} \exp(\mathbf{x}\mathbf{w}_k)} \right)^{[y=c]}$$

- The NLL function simplifies to:

$$\begin{aligned} nll(\mathcal{D}, \theta) &= -\sum_{n=1}^{N} \log P(Y = y_n|\mathbf{X} = \mathbf{x}_n, \theta) \\ &= -\sum_{n=1}^{N} \sum_{c=1}^{C} [y_n = c] \left(\mathbf{x}_n\mathbf{w}_c - \log \left(\sum_{k=1}^{C} \exp(\mathbf{x}_n\mathbf{w}_k) \right) \right) \\ &= -\sum_{n=1}^{N} \left(\left(\sum_{c=1}^{C} [y_n = c]\mathbf{x}_n\mathbf{w}_c \right) - \log \left(\sum_{k=1}^{C} \exp(\mathbf{x}_n\mathbf{w}_k) \right) \right) \end{aligned}$$

Non-Linear Probabilistic Classification

- If we want to build a non-linear probabilistic binary or multi-class classifier, we can use parameter prediction functions based on basis expansions.
- We can also use parameter prediction functions based on an arbitrary neural network model (CNN, MLP, etc.).
- We just need to make sure to apply the logistic transform or the softmax transform to ensure parameters constraints are satisfied.

Making Predictions

- Given a probabilistic supervised model, we can produce an estimate of the conditional probability of y given $\mathbf{x}$ by plugging the estimated parameters $\hat{\theta}$ into the model.
- In the classification case, when we need to issue a prediction, we typically predict the value with the highest probability under the learned model.
- This is the most probable y given the value of $\mathbf{x}$ and the learend parameters $\hat{\theta}$.

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P_{\hat{\theta}}(Y = y | \mathbf{X} = \mathbf{x})$$

Capacity Control

- As with non-probabilistic models, we need to be concerned about underfitting and overfitting with probabilistic models.
- To address underfitting, we can use non-linear models to predict parameters.
- To address overfitting, we can again use methods like early stopping and dropout.
- We can also regularize probabilistic models, leading to a *penalized negative log likelihood* objective function.
- For example, classical binary probabilistic regression with a squared two norm regularizer has the following objective function:

$$pnll(\mathcal{D}, \theta) = -\sum_{n=1}^{N} \log P(Y = y_n | \mathbf{X} = \mathbf{x}_n, \theta) + \lambda \|\mathbf{w}\|_2^2$$

Poisson Random Variables

- Suppose we have a process that produces data such that $z \in \mathbb{Z}^{\geq 0}$.
- One distribution that matches the support of z is the Poisson distribution:

$$P_\lambda(Y = y) = \frac{\lambda^y \exp(-\lambda)}{y!}$$

- This distribution has the constraint that $\lambda \in \mathbb{R}^{>0}$.

Poisson Regression

- Suppose that $\mathcal{Y} = \mathbb{Z}^{\geq 0}$ and $\mathcal{X} \in \mathbb{R}^D$.
- Base Model: $P_\lambda(Y = y) = \frac{\lambda^y \exp(-\lambda)}{y!}$
- Conditional Model: $P_\theta(Y = y|\mathbf{X} = \mathbf{x}) = \frac{f_\theta(\mathbf{x})^y \exp(-f_\theta(\mathbf{x}))}{y!}$
- Parameter Prediction Function: $f_\theta(\mathbf{x}) = \exp(\mathbf{x}\theta)$
- Model Parameters: θ

Poisson Regression

- This gives us the model:

$$P_\theta(Y = y|\mathbf{X} = \mathbf{x}) = \frac{\exp(\mathbf{x}\theta)^y \exp(-\exp(\mathbf{x}\theta))}{y!}$$

- And the NLL simplifies to:

$$\begin{aligned} nll(\mathcal{D}, \theta) &= -\sum_{n=1}^{N} \log P_\theta(Y = y_n|\mathbf{X} = \mathbf{x}_n) \\ &= -\sum_{n=1}^{N} (y_n \mathbf{x}_n \theta - \exp(\mathbf{x}_n\theta) - \log(y_n!)) \end{aligned}$$